

Laboratório de Concorrência · Aula 10

Daniel Mitsuo Tsuyama · RA 24160006-2 · Turma B

Gerado em 09/10/2026, 20:28:45 · tempo registrado no cronômetro: 0:00

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

Sincronização de Cooperação: Ocorre quando uma tarefa depende do resultado ou da conclusão de outra tarefa para poder prosseguir (relação de produtor-consumidor ou precedência). As tarefas trabalham juntas em prol de um objetivo comum, onde uma espera a outra liberar um dado ou recurso.

Exemplo do dia a dia: Uma linha de montagem de carros. A seção de pintura só pode pintar o carro depois que a seção de soldagem terminar de montá-lo.

Sincronização de Competição: Ocorre quando duas ou mais tarefas tentam acessar o mesmo recurso compartilhado e limitado ao mesmo tempo, e esse recurso não pode ser usado simultaneamente sem corromper o estado (exclusão mútua).

Exemplo do dia a dia: Duas pessoas tentando usar o mesmo banheiro em um escritório ao mesmo tempo. A porta trancada (mecanismo de exclusão) garante que apenas uma pessoa por vez use o recurso.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

Esperado teoricamente: 2.000.000 (dois milhões).

1ª execução (n = 1.000.000)

1020500

2ª execução (n = 1.000.000)

1556622

3ª execução (n = 1.000.000)

1329014

Saída do programa

esperado: 2000

obtido: 2000

```

1 // Prática 1 - Condição de corrida (Sebesta 13.2.1, p. 547-549)
2 // Cole em https://onecompiler.com/java
3 public class Main {
4     static int contador = 0; // variável COMPARTILHADA
5
6     public static void main(String[] args) throws InterruptedException {
7         int n = 1000; // repetições de cada thread
8         Runnable incrementa = () -> {
9             for (int i = 0; i < n; i++) contador++;
10        };
11        Thread t1 = new Thread(incrementa);
12        Thread t2 = new Thread(incrementa);
13        t1.start(); t2.start(); // as duas executam ao mesmo tempo
14        t1.join(); t2.join(); // espera as duas terminarem
15        System.out.println("esperado: " + 2 * n);
16        System.out.println("obtido: " + contador);
17    }
18 }
19

```

Console output:

```

esperado: 2000
obtido: 2000

```

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Não bateram com o esperado e foram diferentes em todas as execuções. Eles mudam porque a ordem de execução dos passos das threads (escalonamento gerado pelo sistema operacional/JVM) é não determinística. Dependendo da alternância de contexto da CPU, uma thread sobrescreve o incremento da outra (perda de atualização).

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

O contador está valendo 10.

A Thread A lê o valor 10 para o seu registrador interno.

A Thread B também lê o valor 10 para o seu registrador interno (antes de A gravar de volta).

A Thread A soma 1 no seu registrador ($10 + 1 = 11$).

A Thread B soma 1 no seu registrador ($10 + 1 = 11$).

A Thread A grava o valor 11 na memória compartilhada (contador vira 11).

A Thread B grava o valor 11 na memória compartilhada (contador continua 11).

Duas somas foram feitas, mas o contador só avançou 1 unidade, perdendo uma atualização.

1.3 Com $n = 1000$, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Com $n = 1000$, a execução é tão rápida que as threads frequentemente terminam antes de sofrer preempção (interrupção do escalonador), fazendo com que o erro aparente sumir em testes locais. Isso é perigoso porque bugs de concorrência são "condições de corrida intermitentes": passam em testes com pouca carga (ambiente de desenvolvimento) e quebram de forma catastrófica em produção, com alta carga de usuários.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

AAABBB → 8

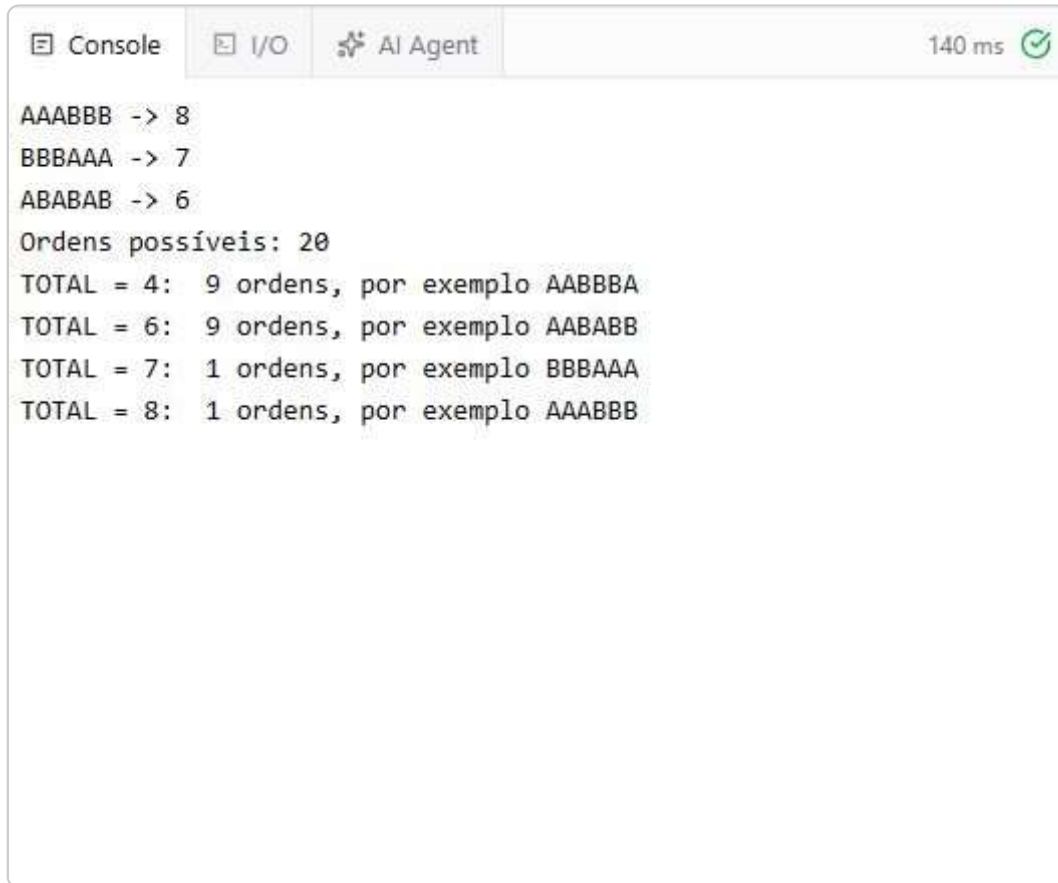
BBBAAA → 7

BABABA → 4

ABABAB → 6

Saída do programa

```
AAABBB → 8
BBBAAA → 7
ABABAB → 6
Ordens possíveis: 20
TOTAL = 4: 9 ordens, por exemplo AABBBB
TOTAL = 6: 9 ordens, por exemplo AABABB
TOTAL = 7: 1 ordens, por exemplo BBBAAA
TOTAL = 8: 1 ordens, por exemplo AAABBB
```



2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

20 ordens são possíveis.

Apenas 01 ordem dá o resultado correto AAABBB

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

A ordem AABBBB resulta em 4.

Passo a passo:

A lê o valor 3 e calcula $3 + 1 = 4$ no seu registrador (mas ainda não salva na memória).

B lê o valor 3 da memória, calcula $3 \times 2 = 6$ e salva o 6 na memória.

Por fim, A salva o seu valor (4) na memória, sobrescrevendo o que B havia gravado.

A operação de B chegou a ser calculada e gravada, mas foi completamente apagada (sobrescrita) pela gravação posterior de A, resultando em 4.

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

Nenhuma das soluções será incorreta; se implementadas corretamente, todas as três abordagens garantem a exclusão mútua ou a atomicidade necessária, eliminando condições de corrida e produzindo o resultado exato na contagem compartilhada.

A mais rápida será a Operação Atômica (AtomicInteger), pois utiliza instruções de hardware de baixo nível (lock-free com CAS) que evitam trocas de contexto e suspensão de threads, superando o overhead de gerenciamento de travas e filas presente no monitor (synchronized) e no semáforo.

Saída do programa

não respondido

```
Console I/O AI Agent
esperado: 2000000
sem sincronizacao: 1046150 em 3 ms
synchronized: 2000000 em 80 ms
AtomicInteger: 2000000 em 73 ms
```

```
Console I/O AI Agent
esperado: 2000000
sem sincronizacao: 1037669 em 2 ms
synchronized: 2000000 em 89 ms
AtomicInteger: 2000000 em 27 ms
```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

As versões synchronized e AtomicInteger.

A versão sem sincronização é mais rápida porque as threads executam de forma totalmente livre e paralela, aproveitando ao máximo o hardware sem nenhum custo de coordenação. Em contrapartida, as versões corretas são mais lentas devido ao overhead de sincronização: elas exigem controle de exclusão mútua, geram contenção de barramento e sincronização de cache entre os núcleos do processador (coerência de cache), e forçam as threads a aguardarem em filas ou repetirem ciclos de processamento, sacrificando o desempenho bruto em troca da integridade dos dados.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Resolve? Não, apenas um AtomicInteger em cada saldo não resolve o problema, pois embora cada saldo individual seja atualizado de forma atômica, a transferência é uma operação composta (retirar de uma conta e colocar na outra) que precisa acontecer junta. Com AtomicInteger isolados, a conta A poderia ser debitada e, no intervalo até a conta B ser creditada, o sistema ficaria em um estado inconsistente e vulnerável a falhas ou condições de corrida. O que seria preciso? Seria preciso garantir a atomicidade de toda a operação de transferência por meio de um mecanismo de travamento (lock) que envolva ambas as contas simultaneamente (como um bloco synchronized abrangente ou transações ACID), assegurando que o débito e o crédito ocorram como um único passo indivisível ou que nenhum deles aconteça em caso de falha.

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

Não, as mensagens "A terminou" e "B terminou" não vão aparecer no fluxo normal de execução.

Isso acontece porque ocorre um impasse (deadlock): a Thread A segura o recurso X e fica esperando o recurso Y, enquanto a Thread B segura o recurso Y e fica esperando o recurso X. Como nenhuma das duas desiste ou libera o recurso que possui, ambas ficam travadas permanentemente esperando a outra liberar.

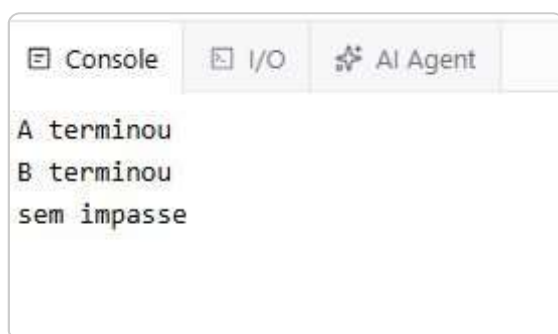
A única forma de o programa não ficar congelado indefinidamente (até o limite de tempo do compilador online) é se ele possuir um mecanismo externo de monitoramento — como o temporizador de 1 segundo mencionado no enunciado que detecta o travamento e força o encerramento da execução.

Saídas (antes e depois da correção)

não respondido



```
Console I/O AI Agent
IMPASSE: 2 threads presas
```



```
Console I/O AI Agent
A terminou
B terminou
sem impasse
```

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

As threads ficaram presas porque a Thread A segurava o recurso X e esperava pelo Y, enquanto a Thread B segurava o recurso Y e esperava pelo X, gerando uma espera circular insolúvel onde nenhuma liberava o que possuía.

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Pegar os recursos na mesma ordem elimina o impasse ao estabelecer uma hierarquia global de bloqueio (como ordenar transferências bancárias sempre pelo menor ID de conta primeiro), o que impede a espera circular e garante que a segunda thread sempre espere o recurso livre antes de travar qualquer coisa.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

Com 3 trabalhadores, espera-se que leve cerca de 200 ms, pois as 6 tarefas são divididas igualmente em duas rodadas de execução paralela (6 dividido por 3 resulta em 2 lotes de 100 ms). Com 1 trabalhador, o tempo seria de 600 ms, já que as tarefas rodariam de forma estritamente sequencial. Com 6 trabalhadores, o tempo cairia para 100 ms, pois todas as tarefas seriam executadas simultaneamente em uma única rodada.

Tempo com 1 trabalhador

600ms

Tempo com 3 trabalhadores

200ms

Tempo com 6 trabalhadores

100ms

Saída do programa

não respondido

```
Console I/O AI Agent
trabalhador 2 fez a tarefa 3
trabalhador 3 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 2 fez a tarefa 4
trabalhador 1 fez a tarefa 6
trabalhador 3 fez a tarefa 5
3 trabalhadores, 6 tarefas de 100 ms: levou 200ms
```

```
Console I/O AI Agent  
trabalhador 1 fez a tarefa 1  
trabalhador 1 fez a tarefa 2  
trabalhador 1 fez a tarefa 3  
trabalhador 1 fez a tarefa 4  
trabalhador 1 fez a tarefa 5  
trabalhador 1 fez a tarefa 6  
1 trabalhadores, 6 tarefas de 100 ms: levou 600ms
```

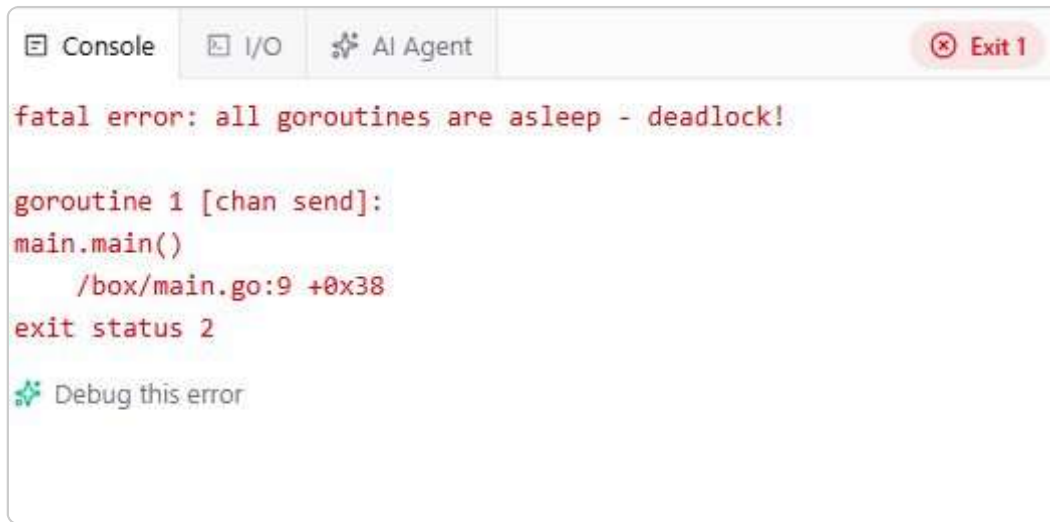
```
Console I/O AI Agent  
trabalhador 5 fez a tarefa 6  
trabalhador 2 fez a tarefa 3  
trabalhador 6 fez a tarefa 1  
trabalhador 1 fez a tarefa 2  
trabalhador 4 fez a tarefa 5  
trabalhador 3 fez a tarefa 4  
6 trabalhadores, 6 tarefas de 100 ms: levou 100ms
```

5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Os tempos variam conforme o paralelismo (600 ms para 1 trabalhador executando de forma sequencial, 200 ms para 3 trabalhadores divididos em duas rodadas, e 100 ms para 6 trabalhadores executando tudo simultaneamente), e o programa não precisa de travas ou synchronized justamente porque adota o modelo de passar mensagens por canais em vez de compartilhar memória diretamente, delegando a sincronização segura dos dados para a própria infraestrutura do canal.

Saídas (antes e depois da troca)

não respondido



The screenshot shows a Go IDE console window with tabs for 'Console', 'I/O', and 'AI Agent'. The console displays a fatal error: 'fatal error: all goroutines are asleep - deadlock!'. Below the error, it shows the stack trace for 'goroutine 1 [chan send]': 'main.main()' at '/box/main.go:9 +0x38', and 'exit status 2'. A 'Debug this error' link is visible at the bottom.

```
fatal error: all goroutines are asleep - deadlock!

goroutine 1 [chan send]:
main.main()
    /box/main.go:9 +0x38
exit status 2

Debug this error
```



The screenshot shows a Go IDE console window with tabs for 'Console', 'I/O', and 'AI Agent'. The console displays a single line of output: '1'. The execution time is shown as '294 ms' with a green checkmark icon.

```
1
```

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

O primeiro programa trava porque um canal sem buffer (unbuffered channel) é síncrono: a instrução de envio (`ch <- 1`) bloqueia a execução da goroutine atual e fica esperando indefinidamente até que outra goroutine faça a leitura (`<-ch`). Como o programa é executado em uma única thread principal e não há outra linha de execução para retirar o valor, o remetente trava para sempre (gerando um deadlock de envio).

O canal com espaço para 1 mensagem (`make(chan int, 1)`) resolve porque ele possui um buffer interno. Isso permite que a operação de envio coloque o valor no canal e prossiga imediatamente sem precisar esperar que alguém o receba naquele exato milissegundo, permitindo que a linha seguinte (`fmt.Println(<-ch)`) seja executada com sucesso.

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

O programa compila e executa com sucesso porque o Rust utiliza o compilador para garantir a segurança em tempo de compilação, permitindo que a thread modifique a variável compartilhada com segurança graças ao uso conjunto do Arc (para múltiplos donos) e do Mutex (para exclusão mútua), o que elimina qualquer risco de condição de corrida antes mesmo de o código rodar.

Saídas (o erro e o resultado com move)

não respondido

Console I/O AI Agent Exit 1

```
error[E0373]: closure may outlive the current function, but it
borrows `contador`, which is owned by the current function
--> HelloWorld.rs:7:27
|
7 |     let t = thread::spawn(|| {
|                               ^^ may outlive borrowed value
`contador`
8 |         contador += 1;           // a thread
altera a variável de main
|         ----- `contador` is borrowed here
|
note: function requires argument type to outlive `static`
--> HelloWorld.rs:7:13
|
7 |     let t = thread::spawn(|| {
|         _____^
8 | |         contador += 1;           // a thread
altera a variável de main
9 | |     });
| |     _____^
help: to force the closure to take ownership of `contador` (and
any other referenced variables), use the `move` keyword
```

```
Console I/O AI Agent 269 ms ✓  
warning: value captured by `contador` is never read  
--> HelloWorld.rs:8:9  
|  
8 |         contador += 1;           // a thread  
altera a variável de main  
|         ^^^^^^^^^  
|  
= help: did you mean to capture by reference instead?  
= note: `#[warn(unused_assignments)]` (part of `#[warn(unused)]`) on by default  
  
warning: value assigned to `contador` is never read  
--> HelloWorld.rs:8:9  
|  
8 |         contador += 1;           // a thread  
altera a variável de main  
|         ^^^^^^^^^^^^^^^^^  
|  
= help: maybe it is overwritten before being read?  
  
warning: 2 warnings emitted  
  
0
```

Saída do programa

não respondido

```
Console I/O AI Agent 447 ms ✓  
contador = 400000
```

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

Com move, o programa compila e imprime o valor correto (400.000) porque o move transfere a propriedade do clone do Arc para a closure de cada thread, permitindo que elas compartilhem o mesmo Mutex na heap de forma segura, enquanto o join garante que o programa espere todas terminarem antes de imprimir.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Enquanto em Java os erros de concorrência e condições de corrida costumam ser descobertos apenas em tempo de execução ou em produção, o Rust intercepta esses problemas estritamente em tempo de compilação por meio de seu sistema de tipos e propriedade; a vantagem é a garantia absoluta de segurança contra bugs de concorrência antes de o código ir ao ar, sendo o custo uma curva de aprendizado muito mais rígida imposta pelo compilador.

Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

As letras serão impressas na ordem A, D, C, B e, após o bloqueio, E, pois o código executa primeiro as instruções síncronas (A e D), depois esvazia a fila de microtarefas (Promises) imprimindo C, passa para a primeira macrotarefa imprimindo B, e por fim executa o segundo setTimeout que, após travar a única thread por 300 ms com o laço de espera, finalmente imprime E com um atraso considerável.

Saída do programa

não respondido



```
▼ Console
A
D
C
B
E: timer de 10 ms rodou depois de 300 ms
```

7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

A ordem de saída foi A, D, C, B (seguida de E). B saiu depois de D e C porque as instruções A e D são síncronas e executam imediatamente. Em seguida, o JavaScript processa as microtasks (como a Promessa de C), que têm prioridade máxima e rodam antes de qualquer fila de temporizadores. Apenas após esvaziar as microtarefas é que a macrotarefa de B (setTimeout com tempo 0) é atendida.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

O timer demorou mais de 300 ms para rodar porque a função ocupado(300) travou a única thread do Node.js com um laço de espera síncrono, impedindo o Event Loop de avançar. Para um servidor Node.js, isso significa que qualquer cálculo pesado ou código síncrono bloqueante paralisa o servidor inteiro, fazendo com que todos os outros usuários fiquem sem resposta até que aquela tarefa pesada seja totalmente concluída.

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O compilador

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

Problema: Pode ocorrer uma condição de corrida em que o sistema lê que há 1 lugar disponível para ambos os usuários ao mesmo tempo, permitindo que as duas transações sejam aprovadas e gerando overselling (venda em duplicidade de um mesmo lugar).

Como evitar: Utilizando controle de concorrência no banco de dados (como transações com bloqueio de exclusão ou isolamento serializável) ou restrições atômicas de atualização na API que garantam que apenas uma requisição consiga decrementar o estoque com sucesso.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais surpreende é ver como linguagens diferentes resolvem o mesmo perigo de formas totalmente opostas — desde o controle estrito exigido pelo compilador do Rust até a elegância da arquitetura sem compartilhamento de memória em Go.